Variables & the Environment

"An idiot admires complexity, a genius admires simplicity" – **Terry Davis**

Hans-Kristian Erik Östlund

Date: **2026.06.11**

Pre-Req: Scala Environment & git (if possible)

Contents

- ❏ **Overview** -> General understanding of the learning goals
- ❏ **PART 1** -> Core concepts and understanding *(15/20 min)*
  - ❏ Inference Rules Review -> Very quick
  - ❏ Interpreters & Programs -> Lettuce Programming Language
  - ❏ Lettuce Lets
  - ❏ Scala Vals
  - ❏ Lets w/ Inference
- ❏ **PART 2** -> Progressive exercises *(20/25 min)*
  - ❏ Focus on implementing a basic lettuce language

When a program uses a variable, how does the interpreter know what value that variable holds?

Part 1

Inference Rules

Inference Rules - Background

- **“Inference Rules” extension of Discrete Mathematics**
  - Propositional logic + Rules of Inference
- **We shall apply logical principles to establish theorems and validate algorithms**
  - These help establish “automated reasoning”

Rules of Inference

Modus Ponens	$\dfrac{p \to q,\ p}{\therefore\ q}$
Modus Tollens	$\dfrac{p \to q,\ \neg q}{\therefore\ \neg p}$
Hypothetical Syllogism	$\dfrac{p \to q,\ q \to r}{\therefore\ p \to r}$
Disjunctive Syllogism	$\dfrac{p \vee q,\ \neg p}{\therefore\ q}$
Addition	$\dfrac{p}{\therefore\ p \vee q}$
Simplification	$\dfrac{p \wedge q}{\therefore\ p}$
Conjunction	$\dfrac{p,\ q}{\therefore\ p \wedge q}$
Resolution	$\dfrac{p \vee q,\ \neg p \vee r}{\therefore\ q \vee r}$

Inference Rules - Notation

Inference Rules Structure

$$\frac{\text{premise}_1, \quad \ldots, \quad \text{premise}_n}{\therefore \ (\text{statement} \ =) \ \text{conclusion}} \ \texttt{rule-name} \qquad n \in \mathbb{Z}^+$$

- **Describe relation between a set of judgements**
- **Axiom:** Statement accepted as true without proof (no premises)
- How do we read this?

Inference Rules - Example (Delete)

- **Delete first instance of n in list**
  - Linked List: L
  - Number: n
- **State Assumptions clearly**
  - Algorithmic design of:
    - Value is not in the list?
    - Is the list ordered?
- **Judgement form: l delete n = v**
- **Concrete Syntax:** Data looks like

Concrete Syntax

$$var \Rightarrow list \mid ERROR$$

$$list \Rightarrow X \mid n \rightarrow list$$

$$n \in \mathbb{N}$$

Inference Rules - Example (Delete)

Inference Rules

$$\frac{}{X\ delete\ n = ERROR}\ \texttt{empty-no-delete}$$

$$\frac{n_1 = n_2}{n_1 \rightarrow list\ delete\ n_2 = list}\ \texttt{non-empty-delete}$$

$$\frac{n_1 \neq n_2, \quad list_{new} = list\ delete\ n_2}{n_1 \rightarrow list\ delete\ n_2 = n_1 \rightarrow list_{new}}\ \texttt{non-empty-tail}$$

- (Explicit vs. Implicit) & Consolidation

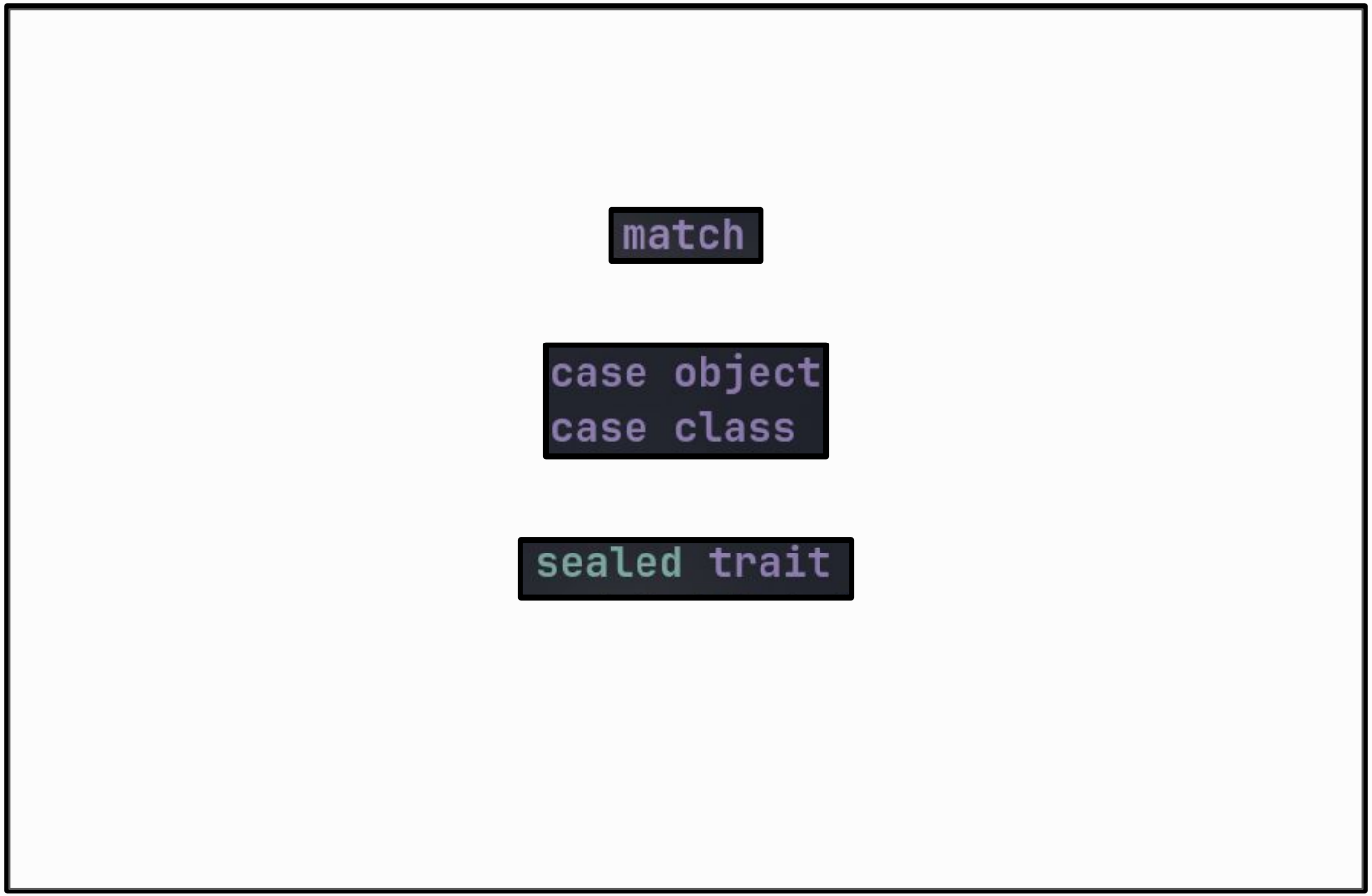
Inference Rules - Example (Delete)

- **Using these Operational Semantics**
  - Assume - l: 5 -> 7 -> 10 -> X
  - Derivation of the list:
    - l
    - => n -> l
    - => 5 -> l
    - => 5 -> n -> l
    - => 5 -> 7 -> l
    - => 5 -> 7 -> n -> l
    - => 5 -> 7 -> 10 -> l
    - => 5 -> 7 -> 10 -> X
- **Now delete 7 from the list**
  - Expect 5 -> 10 -> X

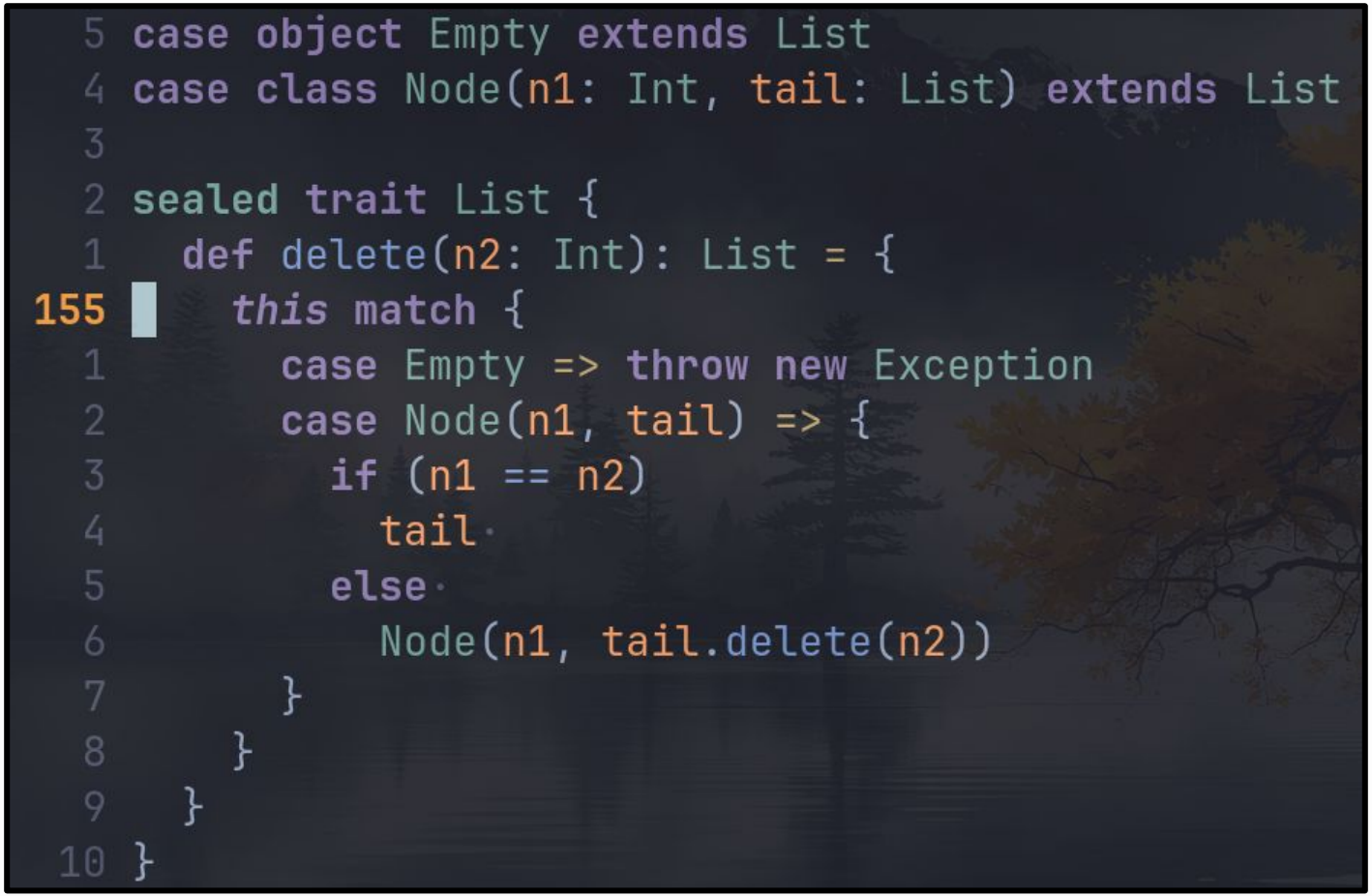
Judgement Derivation

$$\dfrac{5 \neq 7 \qquad \dfrac{7 = 7}{7 \to 10 \to X \; delete \; 7 = 10 \to X}\,\text{non-empty-delete}}{5 \to 7 \to 10 \to X \; delete \; 7 = 5 \to 10 \to X}\,\text{non-empty-tail}$$

Inference Rules - Example (Delete)



Inference Rules - Example (Delete)



```
5  case object Empty extends List
4  case class Node(n1: Int, tail: List) extends List
3
2  sealed trait List {
1    def delete(n2: Int): List = {
155    this match {
1        case Empty => throw new Exception
2        case Node(n1, tail) => {
3          if (n1 == n2)
4            tail
5          else
6            Node(n1, tail.delete(n2))
7        }
8      }
9    }
10 }
```

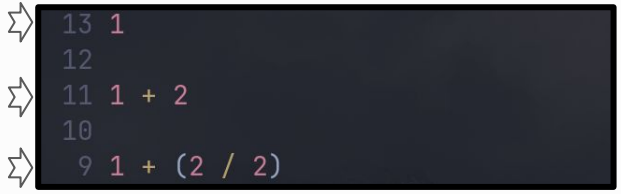
Interpreters & Programs

If we have an expression (e), and values (v), how is this handled?

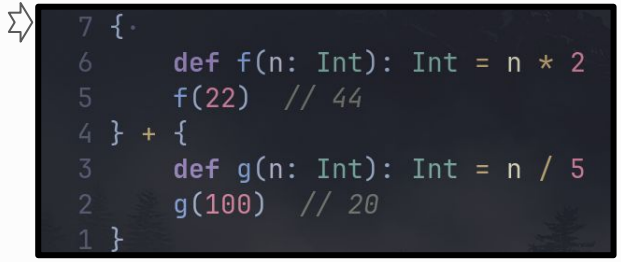
Big-Step / Natural --- eval(1 + 2 * 3) = 7

Scala Vals

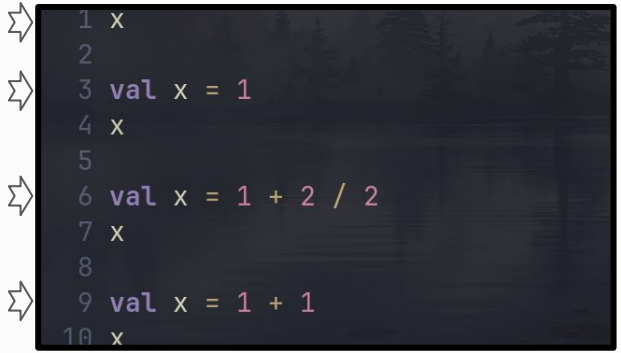
Scala Vals - Expected Values - Scala: Res#: Int = _



```
13 1
12
11 1 + 2
10
 9 1 + (2 / 2)
```

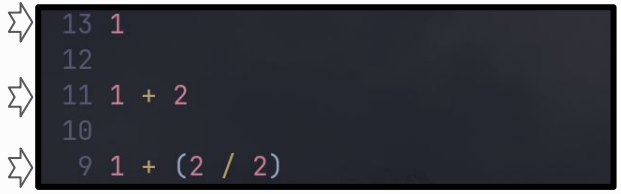


```
7 {
6     def f(n: Int): Int = n * 2
5     f(22)  // 44
4 } + {
3     def g(n: Int): Int = n / 5
2     g(100)  // 20
1 }
```



```
 1 x
 2
 3 val x = 1
 4 x
 5
 6 val x = 1 + 2 / 2
 7 x
 8
 9 val x = 1 + 1
10 x
```

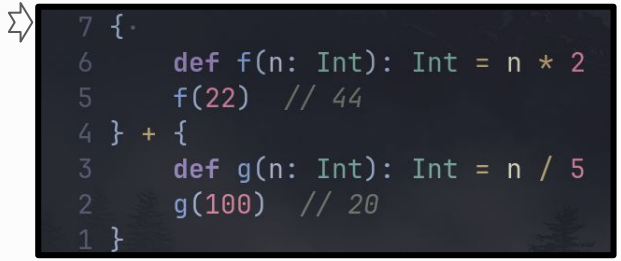
Scala Vals - Expected Values - Scala: Res#: Int = _



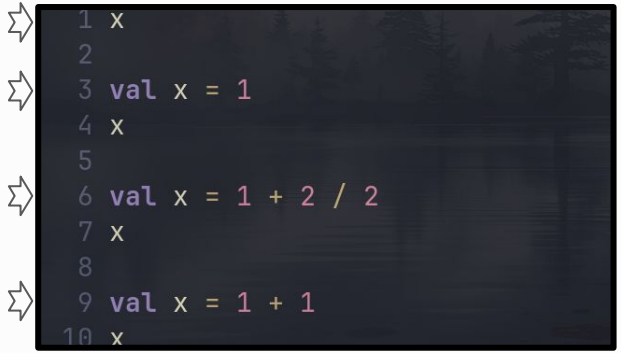
1

3

2



64



Err

1

2

2

Scala Vals - Expected Values - Scala: Res#: Int = _

```
13 1
12
11 1 + 2
10
 9 1 + (2 / 2)
```

1
3
2

```
10 val x = 2
 9 x
 8
 7 2
```

```
 7 {
 6     def f(n: Int): Int = n * 2
 5     f(22)  // 44
 4 } + {
 3     def g(n: Int): Int = n / 5
 2     g(100)  // 20
 1 }
```

64

```
 5 val x = {
 4     val y = 2
 3     y + 1
 2 }
 1 x
```

```
 1 x
 2
 3 val x = 1
 4 x
 5
 6 val x = 1 + 2 / 2
 7 x
 8
 9 val x = 1 + 1
10 x
```

Err
1
2
2

```
 1 val x = {
 2     val y = 2
 3     y + 1
 4 }
 5 x + y
```

Scala Vals - Expected Values - Scala: Res#: Int = _

```
13 1
12
11 1 + 2
10
 9 1 + (2 / 2)
```

1
3
2

```
10 val x = 2
 9 x
 8
 7 2
```

2
2

```
 7 {
 6     def f(n: Int): Int = n * 2
 5     f(22)  // 44
 4 } + {
 3     def g(n: Int): Int = n / 5
 2     g(100)  // 20
 1 }
```

64

```
 5 val x = {
 4     val y = 2
 3     y + 1
 2 }
 1 x
```

3

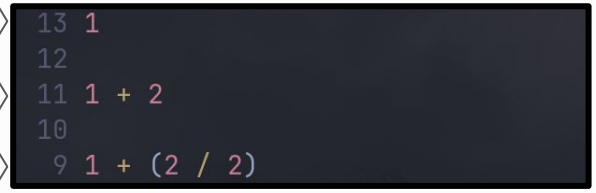
```
 1 x
 2
 3 val x = 1
 4 x
 5
 6 val x = 1 + 2 / 2
 7 x
 8
 9 val x = 1 + 1
10 x
```

Err
1
2
2

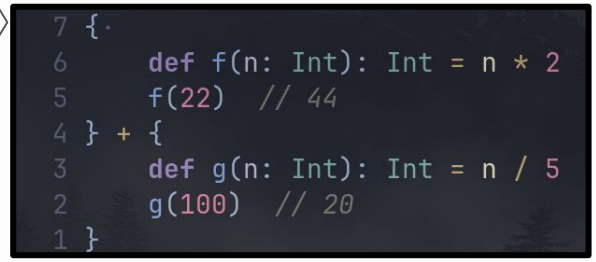
```
 1 val x = {
 2     val y = 2
 3     y + 1
 4 }
 5 x + y
```

Err

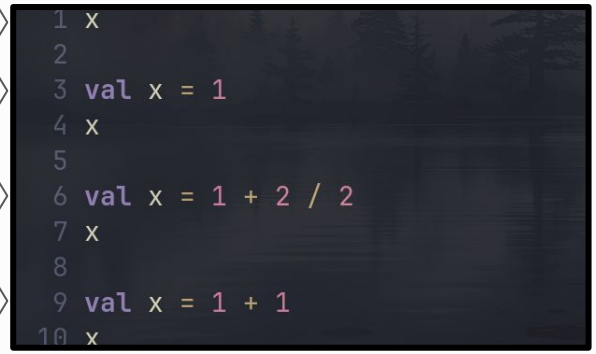
Scala Vals - Expected Values - Scala: Res#: Int = _



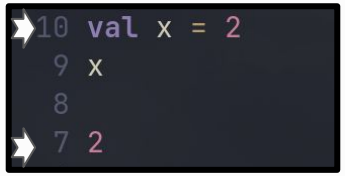
1
3
2



64



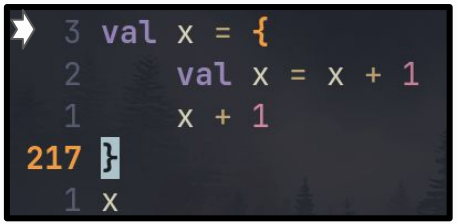
Err
1
2
2

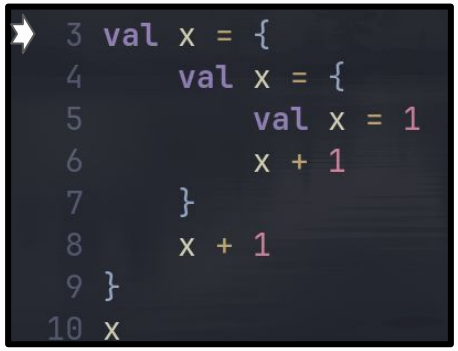


2
2

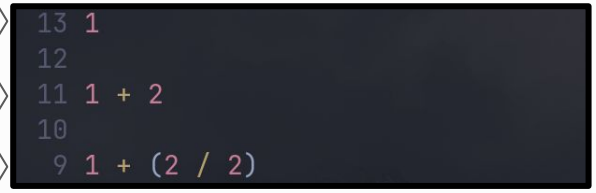
3

Err

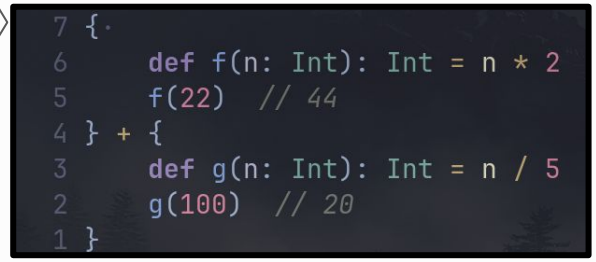




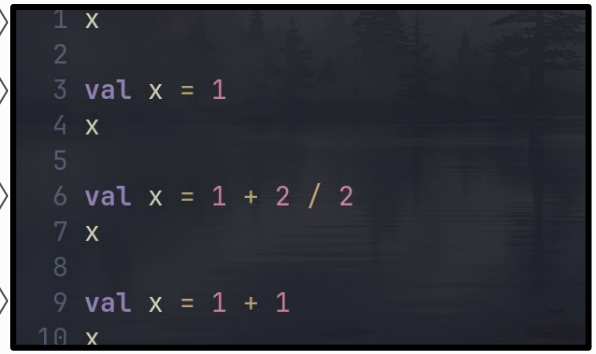
Scala Vals - Expected Values - Scala: Res#: Int = _



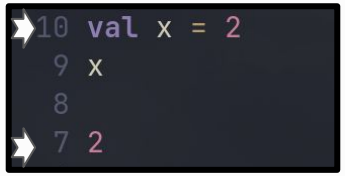
1
3
2



64



Err
1
2
2

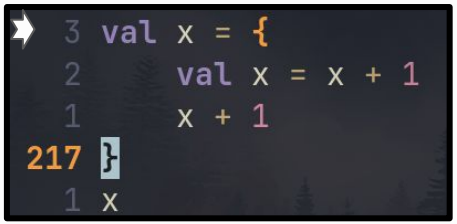


2
2

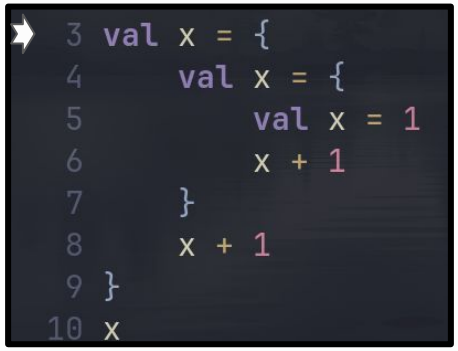
3

Err

3



Err



3

Lettuce Lets

Lettuce Lets - Programming Language

- **We want our own Interpreter**
  - Similar to OCaml
  - Defining language of the program
    - Inductive logic in grammars for concrete syntax
  - Defining semantics in the language
    - Inductive logic as inference rules
  - Using Scala, JVM, ...
    - Rewrite the grammar as abstract syntax
    - Code class Hierarchy
    - Code evaluation methods
    - Any necessary helpers

Example OCaml Code

```
let rec delete n lst =
  match lst with
  | []       -> [] (* delete-empty *)
  | hd :: tl ->
      if hd = n then tl (* delete-head  *)
      else hd :: delete n tl (* delete-tail  *)

let numbers = [1; 2; 3; 4; 3; 5]
let result = delete 3 numbers
(* result = [1; 2; 4; 3; 5] *)
```

Lettuce Lets - Language

Sample Lettuce Concrete Syntax

$$
\begin{array}{lll}
e \Rightarrow v & & \\
\quad \mid e_1 + e_2 & & \text{we can add multiple expressions} \\
\quad \mid e_1 \ / \ e_2 & & \text{we can divide multiple expressions} \\
\quad \mid let\ id = e_1\ in\ e_2 & & \text{we can initialize variables} \\
\quad \mid id & & \text{we can use variables} \\
\quad \mid (e) & &
\end{array}
$$

$$
\begin{array}{lll}
v \Rightarrow n & & \\
\quad \mid ERROR & &
\end{array}
$$

n is a number

id is a variable identifier e.g. x

Lettuce Lets - Similar Expected Value

```
let x =
    let x =
        let x = 1
        in
        x + 1
    in
    x + 1
in
x
```

```
let x =
    let y = 1 + 2/2
    in
    y + 1
in
x
```

$$
\begin{aligned}
e \Rightarrow\ & v \\
& \mid e_1 + e_2 \\
& \mid e_1 \ / \ e_2 \\
& \mid let\ id = e_1\ in\ e_2 \\
& \mid id \\
& \mid (e)
\end{aligned}
$$

$$
\begin{aligned}
v \Rightarrow\ & n \\
& \mid ERROR
\end{aligned}
$$

Our interpreter must know value of variables at each point in time => environment => Map[K,V]

Lets w/ Inference

Lets w/ Inference - Value Domain (old)

Value Inference Rule

$$\frac{v \in \mathbb{N}}{eval(v) = v}\,\texttt{num-ok}$$

- **Totally valid**
  - But what are some issues?
  - Implementation?

Sample Lettuce Concrete Syntax

$$\begin{aligned} e &\Rightarrow v \\ &\mid e_1 + e_2 \\ &\mid e_1 \;/\; e_2 \\ &\mid let\ id = e_1\ in\ e_2 \\ &\mid id \\ &\mid (e) \end{aligned}$$

$$\begin{aligned} v &\Rightarrow n \\ &\mid ERROR \end{aligned}$$

Lets w/ Inference - Value Domain (old)

```scala
sealed trait Expr {
    def eval: Value = this match {
        case N(n) => N(n)
        case _ => ???
    }
}
sealed trait Value extends Expr
case class N(n: Double) extends Value

```

Lets w/ Inference - Value Domain (new)

Value Inference Rule (Rest of Class)

$$\frac{}{eval(Const(n)) = n}\ \texttt{const-ok}$$

- **Allows us to de-couple for method implementations**
  - Notice: Value is no longer an extension of an expression

Lets w/ Inference - Value Domain (new)

```scala
sealed trait Expr {
    def eval: Value = this match {
        case Const(n) => NumValue(n)
        case _ => ???
    }
}
case class Const(f: Double) extends Expr

sealed trait Value
case class NumValue(n: Double) extends Value
```

Part 2

GitHub Repo

Lets w/ Inference - Plus (e1 + e2)

$$\frac{\qquad\qquad}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = \qquad\qquad}\ \texttt{plus-ok}$$

Lets w/ Inference - Plus (e1 + e2) - Now Implement!

$$\frac{eval(e_1) = v_1, \quad v_1 \in \mathbb{Z}, \quad eval(e_2) = v_2, \quad v_2 \in \mathbb{Z}, \quad v_1 + v_2 = v_{res}}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = v_{res}}\texttt{plus-ok}$$

Lets w/ Inference - Gt (e1 > e2)

$$\frac{\qquad\qquad}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = \qquad\qquad}\ \texttt{gt-ok}$$

Lets w/ Inference - Gt (e1 > e2) - Now Implement!

$$\frac{eval(e_1) = v_1, \quad v_1 \in \mathbb{Z}, \quad eval(e_2) = v_2, \quad v_2 \in \mathbb{Z}, \quad v_1 > v_2 = v_{res}}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = v_{res}}\texttt{gt-ok}$$

Lets w/ Inference - Errors?

$$\frac{\square}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = \square}\ \texttt{plus-error-1}$$

$$\frac{\square}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = \square}\ \texttt{gt-error-1}$$

$$\frac{\square}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = \square}\ \texttt{plus-error-2}$$

$$\frac{\square}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = \square}\ \texttt{gt-error-2}$$

Lets w/ Inference - Errors? - Now Implement!

$$\frac{eval(e_1) = v_1, \quad v_1 \notin \mathbb{Z}}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = ERROR}\ \texttt{plus-error-1}$$

$$\frac{eval(e_1) = v_1, \quad v_1 \notin \mathbb{Z}}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = ERROR}\ \texttt{gt-error-1}$$

$$\frac{eval(e_1) = v_1, \quad v_1 \in \mathbb{Z}, \quad eval(e_2) = v_2, \quad v_2 \notin \mathbb{Z}}{eval(\ Plus(\ e_1,\ e_2\ )\ ) = ERROR}\ \texttt{plus-error-2}$$

$$\frac{eval(e_1) = v_1, \quad v_1 \in \mathbb{Z}, \quad eval(e_2) = v_2, \quad v_2 \notin \mathbb{Z}}{eval(\ Gt(\ e_1,\ e_2\ )\ ) = ERROR}\ \texttt{gt-error-2}$$

Lets w/ Inference - Errors? - Now Implement!

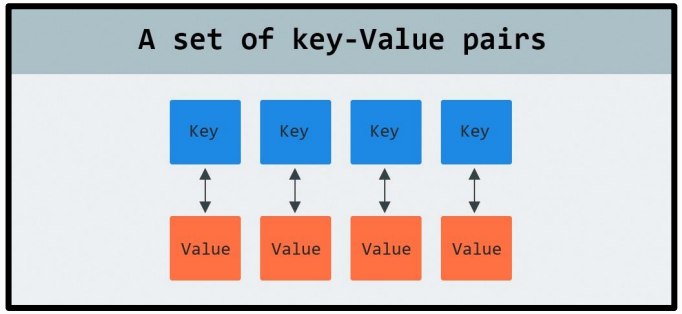
Lettuce does not allow Implicit Type Conversions

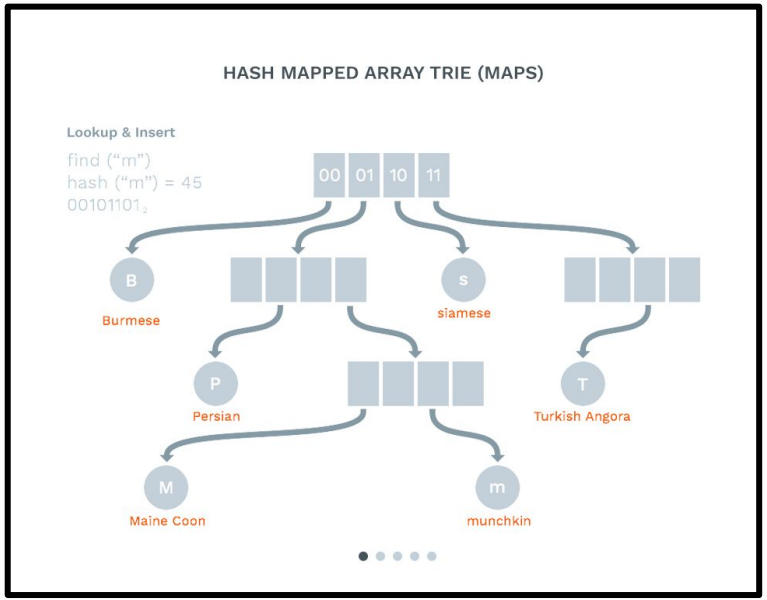
Lets w/ Inference - Scala Map[K,V]

Python: Dict

- **Create:** Map() / Map("x" -> 1.0)
- **Update:** m + (key -> value)
- **Lookup:** m(key) (throws absent)
- **Check Presence:** m.contains(key)
- **Safe Lookup:** m.get(key)

- **At the end of the day**
  - We can store variables for our interpreter





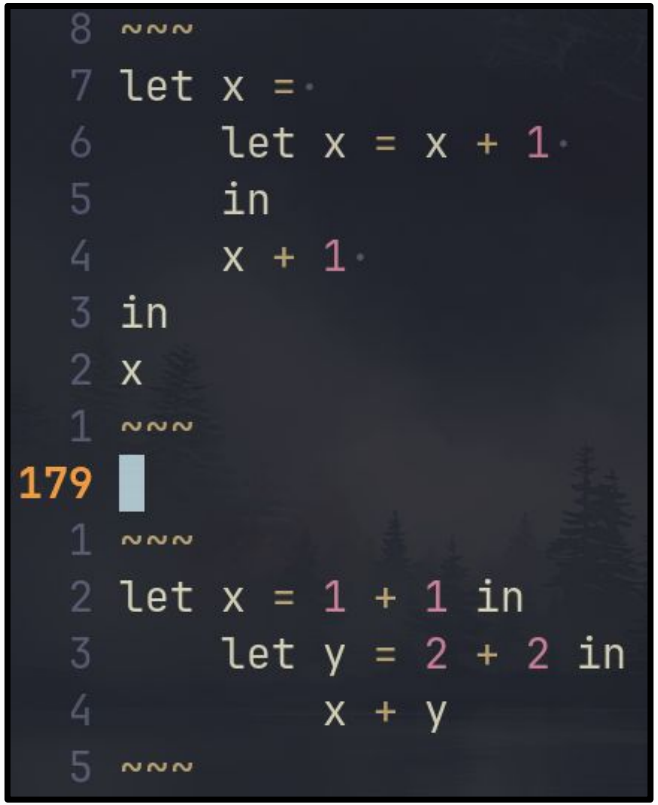
https://blog.dat-ecosystem.org/dat-cryptography/
https://codegym.cc/quests/lectures/en.questsyntaxpro.level13.lecture03

Lets w/ Inference - Scala Map[K,V]

```
11 val m0: Map[String, Double] = Map()
10 val b0: Boolean = m0 contains "x"
 9
 8 val m0: Map[String, Double] = Map()
 7 val m1 = m0 + ("x" -> 3155.0)
 6 val m2 = m1 + ("y" -> 2270.0) + ("z" -> 1300.0)
 5
 4 if (m2.contains("y")) {
 3     val y = m2("y")
 2     println(s"FOUND y: $y")
 1 }
161
 1 // works for other types
 2 val m: Map[Int, String] = Map((2270 -> "Data Structures"), (3155 -> "PPL"))
 3
 4 val k = 3155
 5 if (m contains k) {
 6     val v = m(k)
 7     println(s"FOUND $k: $v")
 8 }
```

Lets w/ Inference - Our Interpreter

Our expression handling goal - eval(e, env) = v



Lets w/ Inference - Our Interpreter - Now Implement!

```
2 type Environment = Map[String, Value]
```

Lets w/ Inference - id and let id = e1 in e2?

$$\frac{\square}{eval(\ Ident(\ id\ ),\ env\ ) = \square}\ \texttt{(ident-error)}$$

$$\frac{\square}{eval(\ Ident(\ id\ ),\ env\ ) = \square}\ \texttt{(ident-ok)}$$

$$\frac{\square}{eval(\ Let(\ id,\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(let-error)}$$

$$\frac{\square}{eval(\ Let(\ id,\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(let-ok)}$$

Lets w/ Inference - id and let id = e1 in e2? - Impl!

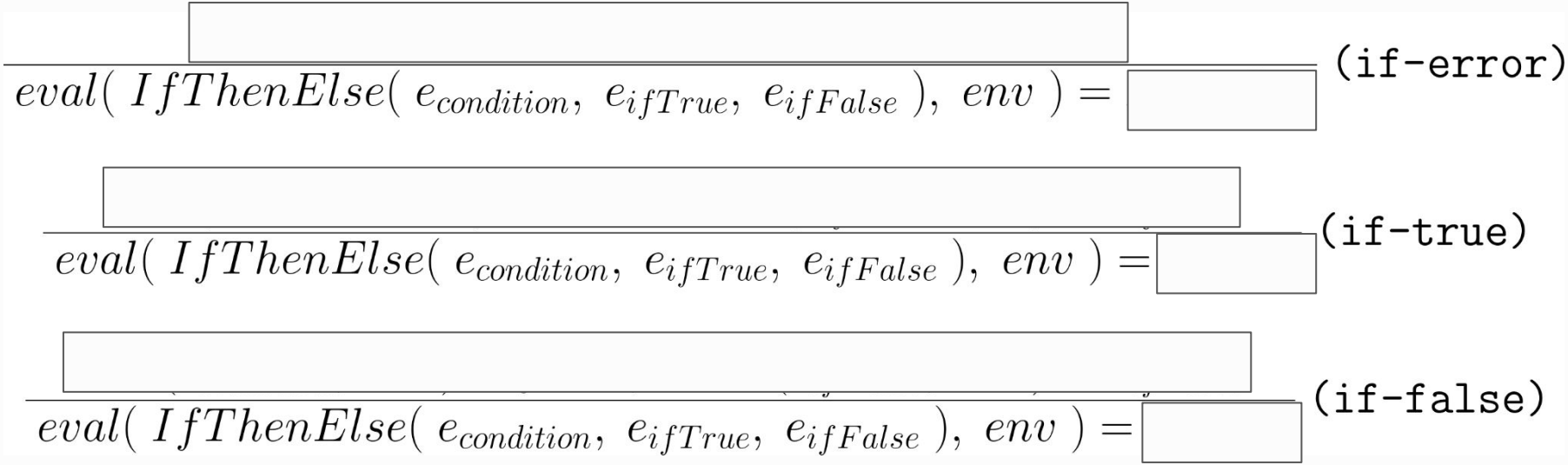
$$\frac{id \notin env}{eval(\ Ident(\ id\ ),\ env\ ) = ERROR}\ \texttt{(ident-error)}$$

$$\frac{id \in env, \quad env(id) = v}{eval(\ Ident(\ id\ ),\ env\ ) = v}\ \texttt{(ident-ok)}$$

$$\frac{eval(e_1,\ env) = ERROR}{eval(\ Let(\ id,\ e_1,\ e_2\ ),\ env\ ) = ERROR}\ \texttt{(let-error)}$$

$$\frac{eval(e_1,\ env) = v_1, \quad v_1 \neq ERROR, \quad env \circ \{id \mapsto v_1\} = env_{new}, \quad eval(e_2,\ env_{new}) = v_2}{eval(\ Let(\ id,\ e_1,\ e_2\ ),\ env\ ) = v_2}\ \texttt{(let-ok)}$$

Lets w/ Inference - if (e1) e2 else e3



Lets w/ Inference - if (e1) e2 else e3 - Impl!

$$\frac{eval(e_{condition},\ env) = v_1, \quad v_1 \notin \{true,\ false\}}{eval(\ IfThenElse(\ e_{condition},\ e_{ifTrue},\ e_{ifFalse}\ ),\ env\ ) = ERROR}\ \texttt{(if-error)}$$

$$\frac{eval(e_{condition},\ env) = true, \quad eval(e_{ifTrue},\ env) = v_{ifTrue}}{eval(\ IfThenElse(\ e_{condition},\ e_{ifTrue},\ e_{ifFalse}\ ),\ env\ ) = v_{ifTrue}}\ \texttt{(if-true)}$$

$$\frac{eval(e_{condition},\ env) = false, \quad eval(e_{ifFalse},\ env) = v_{ifFalse}}{eval(\ IfThenElse(\ e_{condition},\ e_{ifTrue},\ e_{ifFalse}\ ),\ env\ ) = v_{ifFalse}}\ \texttt{(if-false)}$$

Lets w/ Inference - e1 && e2

$$\frac{\square}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(and-error-1)}$$

$$\frac{\square}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(and-false-short-circuit)}$$

$$\frac{\square}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(and-error-2)}$$

$$\frac{\square}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = \square}\ \texttt{(and-true)}$$

- **Short Circuiting**
  - Truth Table of & (TT/TF/FT/FF)
  - F & x = F, T & x = x
  - Avoid looking at x, if don't have to
- Lettuce: only allow BoolValue on && / And(e1, e2)

Lets w/ Inference - e1 && e2 - Impl!

$$\frac{eval(e_1,\ env) = v_1, \quad v_1 \notin \{true,\ false\}}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = ERROR}$$ (and-error-1)

$$\frac{eval(e_1,\ env) = false}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = false}$$ (and-false-short-circuit)

$$\frac{eval(e_1,\ env) = true, \quad eval(e_2,\ env) = v_2, \quad v_2 \notin \{true,\ false\}}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = ERROR}$$ (and-error-2)

$$\frac{eval(e_1,\ env) = true, \quad eval(e_2,\ env) = v_2, \quad v_2 \in \{true,\ false\}}{eval(\ And(\ e_1,\ e_2\ ),\ env\ ) = v_2}$$ (and-true)

Functions

Functions - Basic Understanding

```
let f=function(x)x+1 in
f(5)
```
$\Rightarrow 6$

```
let g=function(y)function(z)y+z in
g(5)(10)
```
$\Rightarrow 15$

```
let x=10 in
let foo=function(y)x+y in
let x=200 in
foo(5)
```
$\Rightarrow 15$